

ЛЕКЦИЯ 10. МУЛЬТИПРОЦЕССОРНЫЕ ОС

В общем случае различают

- **многопроцессорные вычислительные системы (ВС) с общей (разделяемой) памятью** (вычислители с симметричной мультипроцессорной обработкой – **SMP**),
- **ВС с распределенной памятью – многомашинные системы** (вычислители с массовым параллелизмом – **MPP** и **вычислительные кластеры**),
- **распределенные системы** (слабосвязанные системы), объединяемые компьютерной сетью в единую вычислительную систему на прикладном уровне с помощью промежуточного программного обеспечения, предоставляющего однородный уровень для взаимодействующих приложений.

Для организации мультипроцессорной обработки в данных ВС применяют различные инструменты.

В **SMP**-вычислителях мультипроцессорную обработку организуют многопроцессорные ОС.

В **MPP**-вычислителях используются коммуникационные средства многомашинных систем.

В **распределенных системах** применяется промежуточное программное обеспечение, основанное на документе, файлах, объектах, координации.

МНОГОПРОЦЕССОРНЫЕ ОПЕРАЦИОННЫЕ СИСТЕМЫ (МОС)

МОС нацелены на поддержание высокой производительности вычислительных систем с несколькими процессорами. **Основная задача МОС состоит в обеспечении прозрачности числа процессоров для приложений.** Различают три варианта построения МОС.

Простейшая организация МОС основана на принципе: **каждому процессору – своя ОС.** В данной модели статически разделяется память по числу процессоров, каждый процессор имеет свою копию ОС с реентерабельным разделяемым кодом ОС и индивидуальными данными. При такой организации процессоры работают как независимые вычислители.

Достоинствами такого подхода является

- простота,

– возможность заранее запланированного (статического) совместного использования памяти и внешних устройств,

– возможность взаимодействия приложений через общую область памяти.

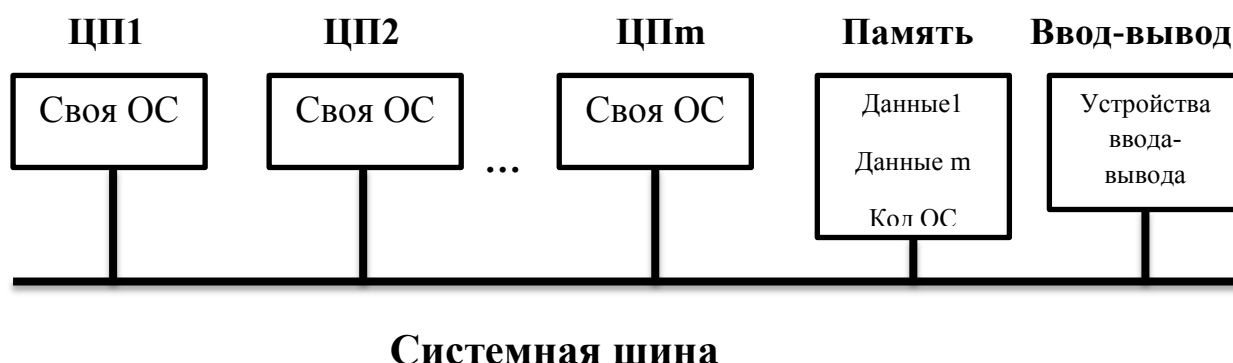
К недостаткам следует отнести:

– дублирование управляющих структур при каждом процессоре,

– невозможность динамического перераспределения используемой памяти,

– невозможность совместного использования ЦП, т.к. каждый процессор планирует свою работу на собственном множестве приложений (как результат – один процессор может быть загружен, а другой – простаивать),

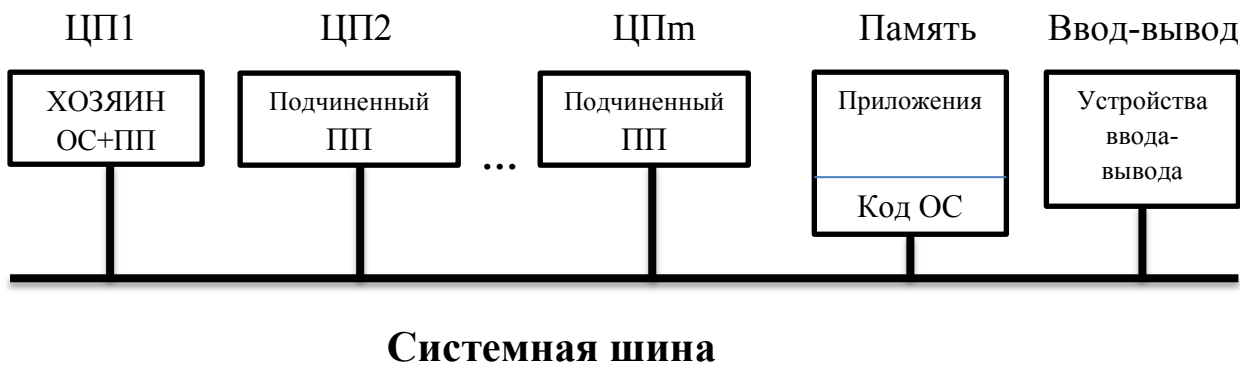
– невозможность обеспечения когерентности кэша данных совместно используемых внешних устройств, поскольку каждый процессор реализует кэширование независимо от других.



В основе второго варианта организации МОС лежит принцип: **хозяин – подчиненный (ведущий – ведомый)**. В данной модели используется одна копия ОС, которая выполняется на первом процессоре вычислителя (ведущий процессор). Все остальные процессоры (ведомые) выполняют прикладные процессы. Все системные вызовы перенаправляются на ведущий процессор, который поддерживает в актуальном состоянии единые управляющие структуры данных. При отсутствии системной работы ведущий процессор также как и остальные выполняет пользовательские приложения.

Основным преимуществом данной организации МОС является отсутствие недостатков предыдущей модели.

Недостатком является потенциально узкое место ведущего процессора при интенсивном потоке системных вызовов от большого числа ведомых процессоров. Такая модель проста и работоспособна при небольшом числе процессоров и применяется обычно во встроенных системах.



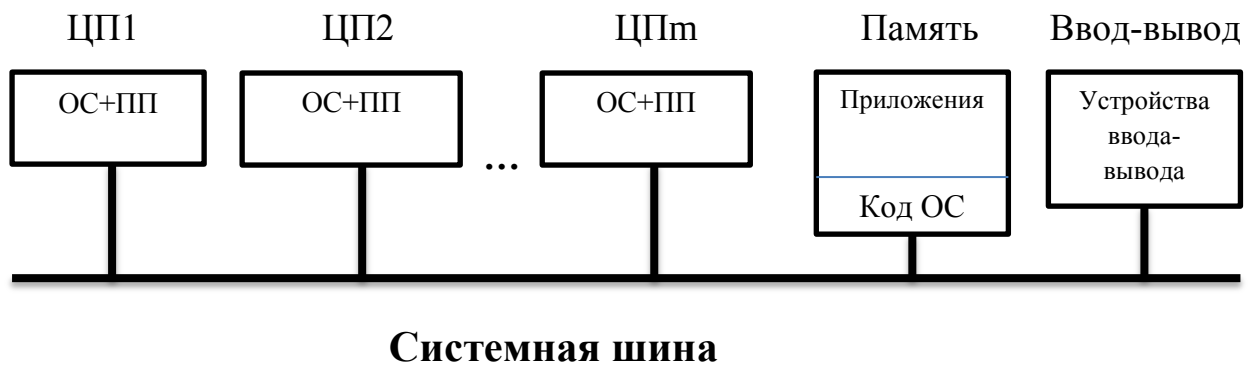
Третья модель МОС основана на **симметричной мультипроцессорной обработке** и устраняет перекося второй модели. Здесь также как и в предыдущей схеме имеется одна копия ОС, но выполнять ее может любой процессор. Системные вызовы от приложений вызывают прерывание на процессоре, выполняющем прикладной процесс с переходом из пользовательского режима в режим ядра и последующей обработкой системного вызова.

В рамках данной модели обеспечивается

- динамический баланс загрузки всех процессоров,
- эффективное использование памяти.

Основной недостаток симметричной МОС состоит в необходимости синхронизации доступа конкурирующих процессоров вычислителя к управляющим структурам.

Для снижения издержек от конкуренции симметричную МОС расщепляют на независимые критические области, не взаимодействующие друг с другом, которые могут выполняться параллельно на различных процессорах. При этом синхронизировать нужно только доступ к конкретным частям МОС. Как следствие код симметричной МОС имеет сложную структуру и требует команды высококвалифицированных специалистов для развития функциональности и сопровождения в течение жизненного цикла. Модель симметричной МОС используется в настоящее время во всех SMP-вычислителях общего назначения.



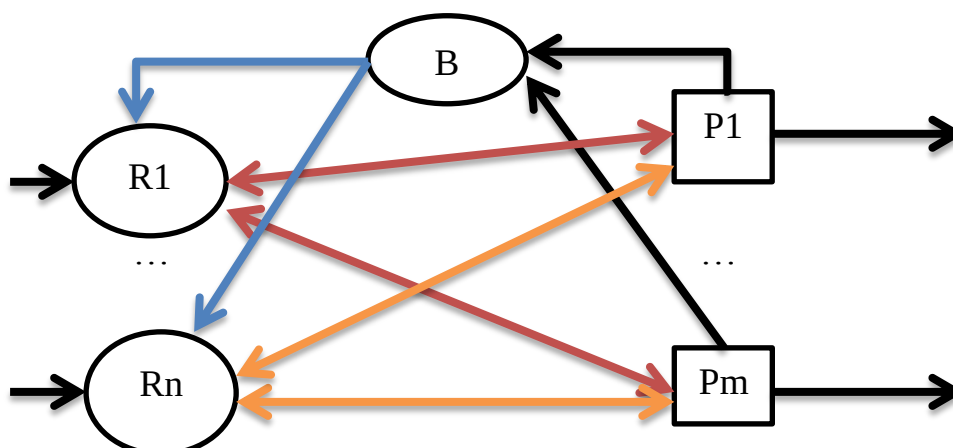
Планирование времени мультипроцессора.

В отличие от однопроцессорной системы, где планирование одномерно, на мультипроцессорном вычислителе планирование двумерно (во времени и в пространстве процессоров). Дополнительным фактором усложняющим планирование является возможная взаимная зависимость связанной группы процессов.

Планирование независимых процессов.

Простейший алгоритм планирования независимых процессов состоит в сопровождении одного или нескольких (по приоритетам) *списков готовых к выполнению процессов*.

- + При этом достигается автоматическая балансировка нагрузки.
- Издержками такого планирования является:
 - высокая конкуренция процессоров за доступ к общим управляющим структурам (прежде всего к очередям готовых процессов),
 - существенные накладные расходы на переключение контекста при блокировании процессов операциями ввода-вывода,
 - значительный объем промахов в кэшах процессоров (и как следствие перемещений блоков памяти и адресных ссылок между кэшем и памятью) при планировании **без предыстории памяти**.

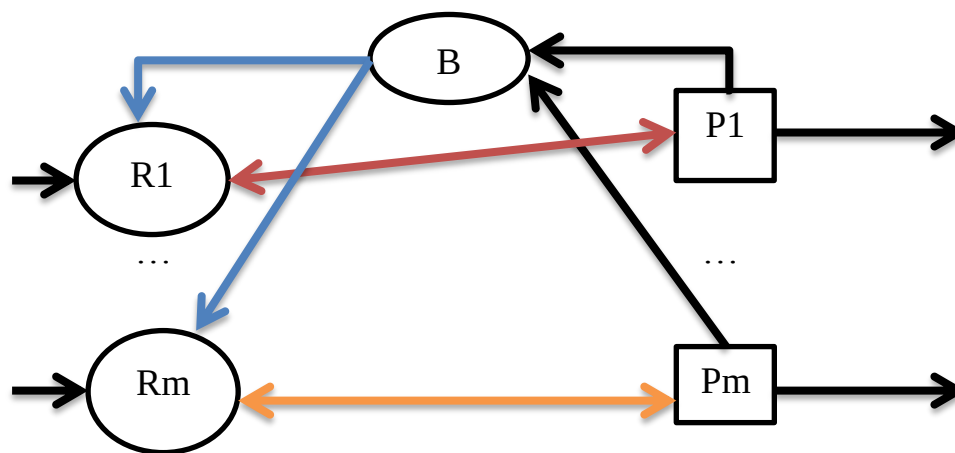


n – приоритетов (очереди готовых процессов), m – процессоров.

Эти недостатки снимаются *родственным планированием* выбора процессов на выполнение. Алгоритм основан на закреплении процессов в течение жизни за определенными процессорами. В момент создания каждый процесс назначается конкретному процессору, при этом **каждый процессор получает свой набор процессов**. Если у какого либо из процессоров отсутствует работа, выполняется перераспределение процессов между процессорами.

Такое планирование в течение жизни процесса обеспечивает

- равномерное распределение нагрузки между процессорами,
- минимизирует конкуренцию за списки процессов (поскольку списки для каждого процессора – свои),
- максимизирует родственность кэша адресных ссылок, а также кэша данных и команд каждого процессора. В силу закрепления процессов за процессорами их кэши, как правило, содержат не вытесненные блоки памяти приложений от предыдущих циклов выделения кванта времени.



n – очередей готовых процессов, m – процессоров.

Планирование времени процессоров для процессов, состоящих из множества связанных потоков.

Простейшим вариантом является *планирование пакетами потоков* только в пространстве процессоров (разделение пространства процессоров). При этом каждый поток выполняется на своем процессоре и удерживает его до завершения потока (в том числе в периоды блокирования потока

операциями ввода-вывода и синхронизации). В каждый момент времени множество процессоров статически разделено на подмножества, в каждом из которых выполняются потоки одного процесса.

Очевидное преимущество такого планирования состоит в устранении многозадачности.

Основным недостатком является существенная потеря времени процессоров при блокировании потоков операциями синхронизации.

Решением данной проблемы является **бригадное планирование** развивающее принцип планирования пакетами потоков на временную ось. При бригадном планировании группы связанных потоков планируются как единое целое – бригада, все члены бригады запускаются одновременно на разных процессорах в режиме разделения времени, все потоки бригады начинают и завершают свои временные интервалы (кванты времени) вместе. В начале каждого кванта все процессоры перепланируются заново, и на каждом процессоре запускается новый поток одной из готовых к выполнению бригад, если какой либо поток блокируется, его процессор простаивает до конца кванта, в течение кванта перепланирование не выполняется. Таким образом, реализуется основная идея бригадного планирования, состоящая в том, чтобы все потоки бригады работали по возможности вместе.

КОММУНИКАЦИОННЫЕ СРЕДСТВА МНОГОМАШИННЫХ СИСТЕМ УРОВНЯ ПОЛЬЗОВАТЕЛЯ.

В системах с массовым параллелизмом наиболее распространены три типа средств: **обмен сообщениями, вызов удаленных процедур, распределенная память совместного доступа**, сохраняющая для программиста модель разработки приложений в едином адресном пространстве разделяемой памяти.

МОДЕЛЬ ПЕРЕДАЧИ СООБЩЕНИЙ построена на парадигме ввода-вывода. Взаимодействие процессов с помощью *обмена сообщениями* организуется, по меньшей мере, двумя операциями (вызовами): *послать* и *принять*. Различают несколько вариантов реализации данных операций.

Первый вариант основан на **блокирующих (синхронных) операциях**, в результате которых процесс, обратившийся к вызову, блокируется до завершения выполнения операции.

Альтернативой блокирующим операциям являются неблокирующие (асинхронные) вызовы, возвращающие управление в вызывающую программу сразу же после запуска операции и обеспечивающие более высокую производительность. Следует отметить, что неблокирующие вызовы приводят к более сложной организации программ.

Неблокирующая операция *послать* может быть реализована тремя способами.

Первое решение основано на копировании отправляемого сообщения в буфер ядра. При этом теряется время процессора на создание дополнительной копии.

Второй вариант предполагает использование прерываний для извещения отправителя о завершении отправки сообщения. В этом случае копирования не требуется, но усложняется организация программы.

Третье решение основано на блокировании «на запись» буфера с отправляемым сообщением до завершения отправки. Данное решение влечет усложнение управления буферным пространством.

Неблокирующий вызов *принять* также имеет три варианта решения.

Первый вариант основан на извещении получателя о получении сообщения с помощью прерываний, что также как и в случае операции *послать* усложняет программу.

Второй вариант предполагает использование почтового ящика для прибывающих сообщений с реализацией процедуры *опроса*, извещающей о поступлении новых сообщений. Основной трудностью при этом является размещение процедуры *опроса* в прикладной программе.

Третье решение основано на активных сообщениях, порождающих временные потоки по обработке прибывающих сообщений. По прибытии сообщения порождается новый (временный) поток в пространстве получающего процесса с указателем на сообщение, который после выполнения заранее определенной процедуры обработки завершается. Основной недостаток решения заключается в значительных накладных расходах на создание и терминирование (завершение) временных потоков.

В модели **КОММУНИКАЦИИ НА ОСНОВЕ ВЫЗОВА УДАЛЕННЫХ ПРОЦЕДУР** вызывающий процесс именуется клиентом, а вызываемый – сервером. По существу взаимодействие процессов в различных узлах многомашинной вычислительной системы сводится к передаче от клиента вызова к серверу и получению от него ответа после исполнения процедуры. Очевидно, что **удаленный вызов процедуры**

должен быть максимально похож на локальный. Однако, несмотря на концептуальную элегантность вызова удаленной процедуры, **локализация клиента и сервера в различных адресных пространствах вносит ряд ограничений.**

Прежде всего, удаленный вызов не допускает передачи параметров через ссылку, а требует явного указания значений параметров. Это может вызвать затруднения в случае необходимости передачи в вызове сложной структуры данных.

Кроме того, в теле процедуры удаленного вызова и клиентского процесса невозможно использование глобальных разделяемых переменных для коммуникации между собой.

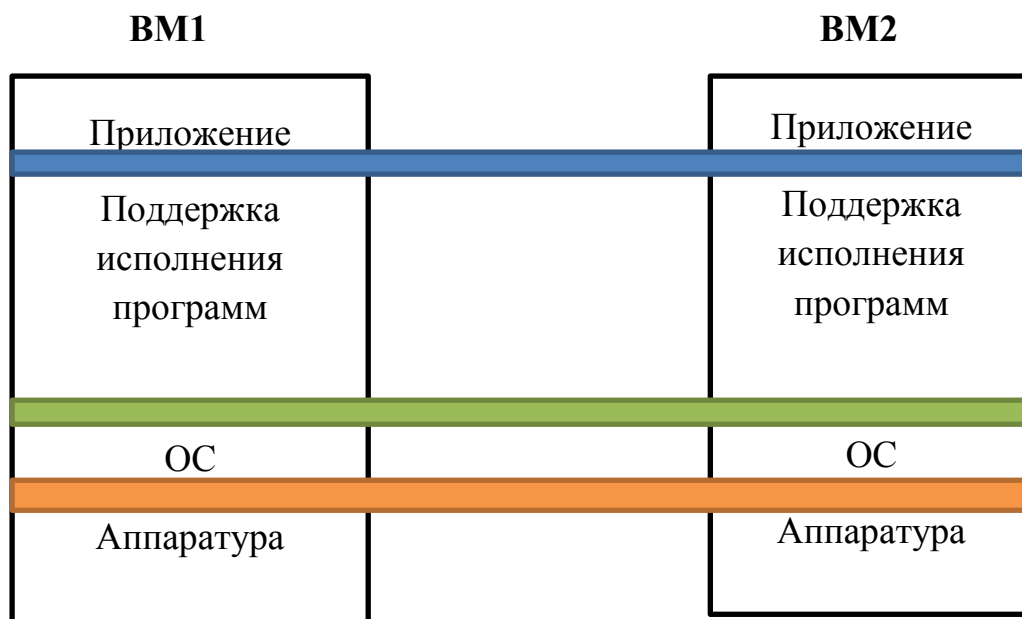
И наконец, реализация удаленного вызова процедур в различных языках программирования требует жесткого мета описания типов и форматов данных, их количества и структуры.

МОДЕЛЬ РАСПРЕДЕЛЕННОЙ ПАМЯТИ СОВМЕСТНОГО ПОЛЬЗОВАНИЯ, реализующая единое адресное пространство, позволяет разрабатывать программы в парадигме общей памяти и переносить на многомашинные платформы традиционные приложения, созданные для SMP-вычислителя. При этом каждый узел вычислителя имеет свою виртуальную память и собственную локальную память.

Отсутствие адресуемого объекта в локальной памяти некоторого узла порождает прерывание, которое удовлетворяется не с локального диска, а по коммуникационной сети с удаленного узла, содержащего данный объект.

Логика организации совместно используемой памяти и единого адресного пространства может быть реализована:

- 1. на аппаратном уровне,**
- 2. на уровне операционной системы,**
- 3. на уровне отдельного, как правило, крупного приложения.**



Основной проблемой данной модели является необходимость обеспечения непротиворечивости копий совместно используемых и изменяемых объектов в различных узлах вычислителя. Реализуется непротиворечивость протоколами когерентности с различными моделями состоятельности.

Поскольку обмен между узлами выполняется блоками памяти значительного размера (для снижения накладных расходов при передаче блоков между узлами), то **проблема непротиворечивости порождает проблему ложного совместного использования** множеством процессов различных несвязанных объектов, расположенных в одном блоке памяти. Очевидно, что чем больше размер блока, тем выше вероятность ложного совместного использования.

КОММУНИКАЦИОННОЕ ПО РАСПРЕДЕЛЕННЫХ СИСТЕМ

Основной целью построения распределенных приложений в распределенных системах (компьютерных сетях) является превращение множества слабосвязанных вычислительных машин в когерентную систему программными средствами. В отличие от многомашинных вычислительных систем узлы территориально распределенной системы:

- связаны существенно слабее (через каналы передачи данных с широким спектром скоростей),
- могут работать на различных технических платформах и под управлением различных ОС,

– могут иметь различные файловые системы (даже под управлением одних и тех же ОС),

– могут иметь различные политики администрирования.

Многомашинные системы включают обычно тысячи узлов, в то время как масштабы распределенной системы могут расширяться до десятков и сотен тысяч узлов.

Сравнение 3-х типов многопроцессорных систем

Аспект	Мультипроцессор	Многомашинная система	Распределенная система
Конфигурация узла	Центральный процессор	Центральный процессор, ОЗУ, сетевой интерфейс	Полноценный компьютер
Периферия узла	Общая, разделяется всеми ЦП	Общая, кроме дисков	Полный набор для каждого узла
Расположение	В одном блоке	В одном помещении	Потенциально по всему миру
Связь между узлами	Общая память	Выделенные линии	Сеть передачи данных
Операционная система	Одна общая	Несколько одинаковых ОС	Могут быть различными
Файловая система	Одна, общая	Одна, общая	У каждого узла своя
Администрирование	Одна организация	Одна организация	Много организаций, у каждого узла своя

С технической точки зрения у мультипроцессоров есть общая память, которой нет у остальных двух многопроцессорных систем. Результатом является использование различных программных моделей. Однако с прикладной точки зрения мультипроцессоры и многомашинные системы представляют собой несколько шкафов с аппаратурой, расположенных в одном помещении. Обе эти системы используются для решения задач, требующих больших объемов вычислений, тогда как распределенная

система, соединяющая компьютеры по интернету, в большей степени занимается связью, чем вычислениями, и используется иначе.

Слабое соединение компьютеров в распределенной системе одновременно является и их силой, и узким местом. **Сильная сторона** состоит в том, что компьютеры могут использоваться для широкого спектра приложений. **Слабость** заключается в том, что программирование таких приложений затрудняется отсутствием лежащей в основе общей модели.

Традиционно в распределенных системах также как и в многомашинных системах можно применять программные парадигмы обмена сообщениями и удаленного вызова процедур. Однако в распределенных системах широкое применение приобрели оригинальные парадигмы построения масштабируемых приложений, основанные на промежуточном программном обеспечении, лежащем над связующим программным обеспечением операционной системы. В целом **распределенные системы стимулировали переход от процессорно-центричной модели компьютера к сетевцентрической модели вычислительных систем.**

Промежуточное ПО, основанное на документе.

Оригинальная парадигма приложения **Всемирная паутина** основана на том, что каждый компьютер может содержать 1 или несколько документов – **web-страниц**. Каждая web-страница может содержать текст, изображения, медиа файлы (звук, видео), и т.д., а также **гиперссылки** (указатели) на др. web-страницы.

Пользовательский интерфейс доступа к web-страницам, рассеянным по узлам сети, реализуется программой – **web-браузером**.

Инструментом доставки web-страниц пользователям обеспечивает протокол **HTTP** (HyperText Transfer Protocol).

Средство создания (разработки) web-страниц реализуется с помощью языка разметки **HTML** (HyperText Markup Language – язык гипертекстовой разметки).

Идентификация web-страниц обеспечивается уникальными сетевыми иерархическими логическими адресами **URL** (Uniform Resource Locator).

Всемирная паутина представляет собой гигантскую коллекцию документов, объединенную с помощью гиперссылок в направленный граф, и работает в парадигме клиент-сервер.

Промежуточное ПО, основанное на файловой системе.

Данный подход состоит в том, чтобы придать распределенной системе вид огромной файловой системы, рассеянной по узлам компьютерной сети. Модель распределенной файловой системы подразумевает, что для связи пользователей (процессов) используется файловый обмен. Один процесс записывает данные в файл, а другой считывает их оттуда. В данной модели возникают следующие проблемы: **модель переноса файлов, именование объектов (файлов), семантика совместного использования файлов.**

МОДЕЛЬ ПЕРЕНОСА ФАЙЛОВ.

Проблема переноса заключается в выборе между **моделью закачивания / скачивания** и **моделью удаленного доступа**.

В **модели закачивания / скачивания** для получения доступа к файлу клиентский процесс сначала полностью считывает его с удаленного сервера (хранящего файл) для обеспечения высокой производительности.

Операции чтения/записи выполняются локально на клиентском компьютере, когда процесс заканчивает работу с файлом, обновленный файл отправляется обратно на сервер.

Преимущество модели:

- простота,
- перенос файла целиком эффективнее, чем перенос его по частям.

Недостатки:

- необходимость наличия достаточно большого объема памяти на клиенте для хранения файла целиком,
- накладные расходы при переносе файла целиком, в то время как требуется только его часть,
- при наличии нескольких конкурирующих пользователей возникает проблема непротиворечивости файлов.

В **модели удаленного доступа** файл остается на сервере, а клиент посылает серверу команды для выполнения работы на самом сервере. Время доступа в этой модели определяется быстродействием сетевого соединения.

ИМЕНОВАНИЕ ОБЪЕКТОВ.

Все распределённые файловые системы поддерживаются каталогами, содержащими файлы. Основной вопрос, возникающий при этом, состоит в том, одинаково ли выглядит иерархия каталогов для всех клиентов.

В распределенных системах различают три общих подхода к именованию файлов и каталогов:

1. **Монтирование** удаленной файловой системы в локальную файловую иерархическую структуру.
2. Имя, задаваемое в виде – **машина+файл**.
3. **Единое пространство имен**, одинаковое на всех машинах распределенной системы.

Именованье файлов в распределенных системах широко опирается на операцию **монтирования**. Системы, работающие с удаленными серверами при помощи монтирования, обладают простотой и гибкостью реализации. Недостатком монтирования является то, что оно не обеспечивает одинаковости вида файловой системы для всех клиентов.

Для обеспечения одинаковости может быть использован глобальный каталог, распознаваемого всеми машинами как корневого. Один из способов поддержки глобального каталога состоит в том, чтобы создать корневой каталог, содержащий только все серверы. При этом все пути к файлам примут одинаковый для всей системы вид **/сервер/путь (машина+файл)**.

Различают две формы прозрачности: **прозрачность местоположения, независимость от местоположения**.

Прозрачность местоположения означает, что по имени пути невозможно определить расположение файла (можно определить сервер, на котором расположен файл, но неизвестно где расположен сам сервер). Сервер может перемещаться по сети без необходимости изменения имени пути.

Независимость от местоположения означает, что файлы могут перемещаться с одного сервера на другой без изменения пути файла. Независимость пути файла от местоположения сложно реализовать, но это свойство очень ценно для распределенных прикладных систем.

Семантика совместного использования файлов.

При использовании файла несколькими пользователями необходимо определить семантику чтения/записи файла для предотвращения возможных проблем. В однопроцессорных системах все операции в/в упорядочиваются естественным образом в единую последовательность, одинаковую для всех процессов. Такая модель обладает **последовательной непротиворечивостью**.

В распределенной системе последовательная непротиворечивость может быть легко достигнута при наличии всего одного сервера. Тогда все операции чтения-записи поступают напрямую на файловый сервер, который обрабатывает их в строгой последовательности. Но один файловый сервер ограничивает производительность. Проблема повышения производительности решается кэшированием (копированием часто используемых файлов в локальной памяти). Но кэширование порождает **проблему когерентности** при совместном использовании одного файла несколькими пользователями.

Возможные решения:

1. **Немедленное распространение всех изменений** кэшируемых файлов обратно на сервер.
2. **Сеансовая семантика**, ослабляющая семантику последовательно непротиворечивого совместного использования файлов. Согласно сеансовой семантики изменения всех открытых файлов сначала видны только процессу, выполнившему эти изменения (хозяину). После закрытия файла изменения видны другим процессам. При использовании и модификации файла одновременно несколькими клиентами необходимо доопределять семантику.
3. **Модель закидывания-скачивания с автоматической блокировкой** скачанных файлов. Попытки всех остальных клиентов получить уже считанный кем-то файл будут приостановлены до того момента, пока первый клиент не вернет файл. При повышенном спросе на файл сервер может посылать клиенту, удерживающему файл, сообщения об ускорении возврата файла.

В целом следует отметить, что составление корректной семантики использования общих файлов является трудной задачей, не имеющей элегантных и эффективных решений.

Промежуточное ПО, основанное на совместно используемых объектах.

В рамках данной парадигмы распределенная система оперирует не с документами и файлами, а с **ОБЪЕКТАМИ**. **Объект – набор переменных, объединенных вместе с набором процедур доступа к ним, называемых методами.** Процессам не разрешается получать доступ к переменным напрямую. Процессы должны для этого вызывать методы.

Примером систем, основанных на объектах, является технология CORBA (Common Object Request Broker Architecture – архитектура распределенных объектных приложений). Технология CORBA представляет

собой систему типа клиент-сервер, в которой клиентский процесс может осуществлять операции с объектами, расположенными на серверах.

Архитектура CORBA разработана для неоднородных аппаратных и программных платформ. Согласование разно платформенных клиента и сервера реализуется через посредников ORB (Object Request Broker – брокер объектных запросов). Каждый объект системы CORBA описывается на специальном языке описания интерфейсов IDL (Interface Definition Language).

В определении объекта указываются методы, экспортируемые объектом, и типы параметров для каждого метода. При создании объекта CORBA создается ссылка на этот объект, которая возвращается процессу, создавшему объект. В дальнейшем процесс обращается к методам созданного объекта по этой ссылке. Ссылка может быть передана другим процессам или сохранена в объектной библиотеке.

Чтобы вызвать метод объекта, клиентский процесс сначала должен получить ссылку на этот объект. Ссылку можно получить либо напрямую от создавшего объект процесса, либо при помощи поиска этого объекта или его метода по имени в специальном каталоге. Когда ссылка на объект получена, клиентский процесс упаковывает параметры вызываемого метода в стандартную структуру и связывается с клиентским ORB-посредником. Тот, в свою очередь, посылает сообщение серверному ORB-посреднику, который вызывает метод объекта. Весь механизм схож с вызовом удаленной процедуры RPC.

Задача ORB-посредников заключается в сокрытии всех низкоуровневых деталей связи и распределения от клиентской и серверной программ. В частности ORB-посредники скрывают от клиента расположение сервера, аппаратное обеспечение ORB сервера, информацию о том, является ли сервер двоичной программой или сценарием, является ли объект активным в настоящий момент, способ общения двух ORB-посредников (TCP/IP, RPC, общая память и т.д.).

Для связи через интернет между клиентским и серверным ORB-посредниками используется протокол IIOP (Internet InterORB Protocol). Для обеспечения возможности использования в системах CORBA объектов, не предназначенных специально для этого, каждый объект оснащается адаптером объекта (упаковщик, обрабатывающий рутинные операции типа формирования ссылок на объект, активация объекта в случае, если тот при вызове находится в неактивном состоянии).

Основной недостаток системы CORBA заключается в том, что каждый объект расположен только на одном сервере, что означает низкую производительность для объектов, одновременно используемых многими клиентами. Обычно архитектура CORBA приемлемо работает только в небольших системах (например, в пределах локальной сети).

Промежуточное ПО, основанное на координации.

Данный вид интерфейсного слоя программного обеспечения для создания приложений основан на парадигме **публикация/подписка**. В рамках этой модели набор процессов, объединенных сетью, может порождать и потреблять информацию.

При появлении у процесса новой информации он ширококестельно публикует иерархический каталог ее тем (рассылает ее по сети всем процессам-партнерам). Процессы, которых интересуют определенные темы (читатели), могут подписаться на них. Чтобы подписаться на какую-либо тему, нужно сообщить ее специальному демону-подписчику (подписаться), работающему на машине процесса-подписчика.

При ширококестельной рассылке (публикации) издателем сводки нового набора информационных сообщений демон-подписчик на каждой машине селектирует из набора сообщения только на подписанные темы. Модель **публикация/подписка** полностью отделяет (исключает обратную связь) производителей информации (издателей) от потребителей (подписчиков). Очень часто эта модель применяется в современном интернете вещей.